INFOSYS SE ASSESSMENT 2026 • OFFICIAL PREPARATION VAULT

# Topic 04: Pseudo Code Compendium (35 Questions)

**Section:** 04_Pseudocode_and_Technical | **Drive Date:** 5th & 6th November 2026 | **Status:** 100% Solved & Verified

## Topic 04: Pseudo Code Compendium (35 Questions)

### Infosys System Engineer ITM Mock Test Series (Papers 01 to 07)

> **Curriculum Overview:** Section 4 tests algorithmic thinking, recursion, bit manipulation (Kernighan's algorithm, bit shifts, XOR/AND masking), nested loop flow control (break/continue nuances), data structures (stack LIFO, queue FIFO), and dynamic programming array transitions.
> **Total Questions:** 35 Questions (5 Questions per Paper × 7 Papers)
> **Target Time:** 10 Minutes per 5-question section (2.0 min/question)
> **Verification Status:** 100000000% Verified Solutions & Zero Hallucinations.

---

#### Question ITM-P01-Q46 (Infosys System Engineer ITM Mock Paper 1 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 5
**Visual Reference Plate:** paper_01_page_05.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
int f(int n) {
    if (n <= 1) return n;
    return f(n-1) + 2 * f(n-2);
}
print(f(6))
```

**Options:**
- **(A)** 15
- **(B)** 21
- **(C)** 27
- **(D)** 31

**Correct Answer:** Option **(B)** — `21`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
Recursive function trace:
- f(0) = 0
- f(1) = 1
- f(2) = f(1) + 2·f(0) = 1 + 0 = 1

- f(3) = f(2) + 2·f(1) = 1 + 2(1) = 3
- f(4) = f(3) + 2·f(2) = 3 + 2(1) = 5
- f(5) = f(4) + 2·f(3) = 5 + 2(3) = 11
- f(6) = f(5) + 2·f(4) = 11 + 2(5) = 21.

---

## Question ITM-P01-Q47 (Infosys System Engineer ITM Mock Paper 1 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_01_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
x = 45; count = 0;
while (x != 0) {
    x = x & (x - 1);
    count++;
}
print(count ^ (45 >> 2))
```

**Options:**
- **(A)** 4
- **(B)** 11
- **(C)** 15
- **(D)** 7

**Correct Answer:** Option **(C)** — `15`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
Brian Kernighan's bit-counting loop clears the lowest set bit in each iteration.
For x = 45 = 32 + 8 + 4 + 1 = $101101_2$ (4 set bits):
Loop executes 4 times => count = 4.
Right shift: 45 >> 2 = 11 (binary $1011_2$).
Bitwise XOR: count ^ 11 = 4 ^ 11 = $0100_2$ ^ $1011_2$ = $1111_2$ = 15.

---

## Question ITM-P01-Q48 (Infosys System Engineer ITM Mock Paper 1 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_01_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
s = 0
for i = 1 to 5
    for j = i to 5
        if ((i + j) % 3 == 0) continue
        if (j == 5) break
        s = s + i * j
    end for
end for
print(s)
```

**Options:**
- **(A)** 38
- **(B)** 42
- **(C)** 46
- **(D)** 50

**Correct Answer:** Option **(C)** — `46`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
Nested loops simulation:
- i=1: j=1 (sum=2, add 1); j=2 ((1+2)%3==0, continue); j=3 (sum=4, add 3); j=4 (sum=5, add 4); j=5 ((1+5)%3==0, continue before break!). Running s = 1 + 3 + 4 = 8.
- i=2: j=2 (sum=4, add 4); j=3 (sum=5, add 6); j=4 ((2+4)%3==0, continue); j=5 (j==5, break). Running s = 8 + 4 + 6 = 18.
- i=3: j=3 ((3+3)%3==0, continue); j=4 (sum=7, add 12); j=5 (j==5, break). Running s = 18 + 12 = 30.
- i=4: j=4 (sum=8, add 16); j=5 ((4+5)%3==0, continue before break). Running s = 30 + 16 = 46.
- i=5: j=5 (j==5, break).
Final s = 46.

---

## Question ITM-P01-Q49 (Infosys System Engineer ITM Mock Paper 1 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_01_page_06.png

**Pseudocode / Algorithm Stem:**

```
A stack is used (operands are evaluated left to right). What is printed?
push(3); push(5); push(7);
x = pop();
push(x * 2);
push(pop() + pop());
print(pop() - pop())
```

**Options:**
- **(A)** 14
- **(B)** 16
- **(C)** 19
- **(D)** 22

**Correct Answer:** Option **(B)** — `16`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
Stack execution trace (LIFO):
1) push(3) -> [3]
2) push(5) -> [3, 5]
3) push(7) -> [3, 5, 7]
4) x = pop() => x = 7; stack = [3, 5]
5) push(7 * 2) = push(14) -> [3, 5, 14]
6) push(pop() + pop()): first pop() returns 14, second pop() returns 5; 14 + 5 = 19; push(19) -> [3, 19]
7) print(pop() - pop()): first pop() returns 19, second pop() returns 3; 19 - 3 = 16.

---

## Question ITM-P01-Q50 (Infosys System Engineer ITM Mock Paper 1 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_01_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is the final content of arr? (indexing starts at 0; updates are in place)
arr = [4, 1, 3, 2]
for i = 0 to 3
    arr[i] = arr[i] + arr[arr[i] % 4]
end for
```

**Options:**
- **(A)** [8, 2, 5, 7]
- **(B)** [8, 2, 5, 6]
- **(C)** [8, 5, 5, 7]
- **(D)** [5, 2, 5, 7]

**Correct Answer:** Option **(A)** — `[8, 2, 5, 7]`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
In-place array update with arr = [4, 1, 3, 2]:
- i = 0: arr[0] = arr[0] + arr[arr[0] % 4] = 4 + arr[4 % 4] = 4 + arr[0] = 4 + 4 = 8. arr = [8, 1, 3, 2].
- i = 1: arr[1] = arr[1] + arr[arr[1] % 4] = 1 + arr[1 % 4] = 1 + arr[1] = 1 + 1 = 2. arr = [8, 2, 3, 2].
- i = 2: arr[2] = arr[2] + arr[arr[2] % 4] = 3 + arr[3 % 4] = 3 + arr[3] = 3 + 2 = 5. arr = [8, 2, 5, 2].
- i = 3: arr[3] = arr[3] + arr[arr[3] % 4] = 2 + arr[2 % 4] = 2 + arr[2] = 2 + 5 = 7. arr = [8, 2, 5, 7].
Final content: [8, 2, 5, 7].

---

## Question ITM-P02-Q46 (Infosys System Engineer ITM Mock Paper 2 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 5
**Visual Reference Plate:** paper_02_page_05.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
int f(int n) {
    if (n == 0) return 1;
    if (n == 1) return 2;
    return f(n-1) + f(n-2) * 2;
}
print(f(5))
```

**Options:**
- **(A)** 26
- **(B)** 30
- **(C)** 32
- **(D)** 34

**Correct Answer:** Option **(C)** — `32`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
32. f(0)=1, f(1)=2, f(2)=2+2=4, f(3)=4+4=8, f(4)=8+8=16, f(5)=16+16=32.

---

## Question ITM-P02-Q47 (Infosys System Engineer ITM Mock Paper 2 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 5
**Visual Reference Plate:** paper_02_page_05.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (x and y are unsigned integers)
x = 58; y = 0;
while (x > 0) {
    y = (y << 1) | (x & 1);
    x = x >> 1;
}
print(y)
```

**Options:**
- **(A)** 23
- **(B)** 29
- **(C)** 37
- **(D)** 41

**Correct Answer:** Option **(A)** — `23`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
23. The loop peels off bits of x from the LSB and rebuilds y by shifting left each time, which reverses the bit order of x within its significant bits. 58 = $111010_2$ (6 bits); reversed = $010111_2$ = 23. (Trace: y goes 0→0→1→2→5→11→23 as x's bits 0,1,0,1,1,1 are consumed in order.)

---

## Question ITM-P02-Q48 (Infosys System Engineer ITM Mock Paper 2 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 5
**Visual Reference Plate:** paper_02_page_05.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
s = 0
for i = 1 to 4
    for j = 1 to 4
        if (i == j) continue
        if (i + j > 6) break
        s = s + (i * j)
    end for
end for
print(s)
```

**Options:**
- **(A)** 40
- **(B)** 44
- **(C)** 46
- **(D)** 48

**Correct Answer:** Option **(C)** — `46`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
46. i=1: skips j=1 (i==j); adds 1×2+1×3+1×4 = 9. i=2: skips j=2; adds 2×1+2×3+2×4 = 16 (running 25; note j=4 gives i+j=6, which is not >6, so it's included). i=3: skips j=3; adds 3×1+3×2 = 9, then j=4 gives i+j=7>6
→ break (running 34). i=4: adds 4×1+4×2 = 12, then j=3 gives i+j=7>6 → break (running 46).

---

## Question ITM-P02-Q49 (Infosys System Engineer ITM Mock Paper 2 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 5
**Visual Reference Plate:** paper_02_page_05.png

**Pseudocode / Algorithm Stem:**

```
A queue is used (FIFO). What is printed?
enqueue(2); enqueue(4); enqueue(6);
a = dequeue();
enqueue(a + 10);
b = dequeue();
c = dequeue();
print(b + c)
```

**Options:**
- **(A)** 10
- **(B)** 16
- **(C)** 18
- **(D)** 22

**Correct Answer:** Option **(A)** — `10`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
10. Queue [2,4,6] → dequeue 2 (a) → [4,6] → enqueue 12 → [4,6,12] → dequeue 4 (b) → dequeue 6 (c) → [12]. b + c = 4 + 6 = 10.

---

## Question ITM-P02-Q50 (Infosys System Engineer ITM Mock Paper 2 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_02_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is the final content of arr? (0-indexed; each statement uses the array's state at that point)
arr = [1, 2, 3, 4, 5]
temp = arr[0]
arr[0] = arr[4]
arr[4] = arr[2]
arr[2] = arr[1]
arr[1] = temp
```

**Options:**
- **(A)** [5, 1, 2, 4, 3]
- **(B)** [5, 1, 3, 4, 2]
- **(C)** [5, 2, 1, 4, 3]
- **(D)** [5, 1, 2, 4, 2]

**Correct Answer:** Option **(A)** — `[5, 1, 2, 4, 3]`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
temp=1. arr[0]=arr[4]=5 → [5,2,3,4,5]. arr[4]=arr[2]=3 → [5,2,3,4,3]. arr[2]=arr[1]=2 → [5,2,2,4,3]. arr[1]=temp=1 → final [5,1,2,4,3].

---

## Question ITM-P03-Q46 (Infosys System Engineer ITM Mock Paper 3 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 5
**Visual Reference Plate:** paper_03_page_05.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
```

**Options:**
- **(A)** 40
- **(B)** 46
- **(C)** 52
- **(D)** 44

int f(int n) {
if (n <= 0) return 0;
if (n == 1) return 1;

```
return f(n-1) + f(n-2) + n;
}
print(f(6))
```

```
x = 37; mask = 0;
for i = 0 to 5
if (((x >> i) & 1) == 1)
mask = mask | (1 << (5 - i))
end if
end for
print(mask)
s = 0; i = 1
while (i <= 6)
j = 1
while (j <= i)
if (j % 2 == 0)
j = j + 1
continue
end if
s = s + (i * j)
j = j + 1
end while
i = i + 1
end while
print(s)
push(4); push(7);
a = pop(); b = pop();
push(a - b);
push(10);
c = pop(); d = pop();
push(c + d);
push(3);
e = pop(); f = pop();
print(e * f)
arr = [10, 20, 30, 40, 50, 60]
i = 0; j = 5
while (i < j)
temp = arr[i]
arr[i] = arr[j]
arr[j] = temp
i = i + 1
j = j - 2
end while
print(arr)
```

**Correct Answer:** Option **(B)** — `46`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
46. f(0)=0, f(1)=1, f(2)=1+0+2=3, f(3)=3+1+3=7, f(4)=7+3+4=14, f(5)=14+7+5=26, f(6)=26+14+6=46.

---

## Question ITM-P03-Q47 (Infosys System Engineer ITM Mock Paper 3 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_03_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
```

**Options:**
- **(A)** 37
- **(B)** 41
- **(C)** 45
- **(D)** 39

**Correct Answer:** Option **(B)** — `41`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
41. 37 = $100101_2$ (bits: bit0=1,bit2=1,bit5=1, rest 0). The loop places bit i of x into position (5−i) of mask, which reverses the 6-bit pattern: 100101 reversed is 101001 = 32+8+1 = 41.

---

## Question ITM-P03-Q48 (Infosys System Engineer ITM Mock Paper 3 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_03_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
```

**Options:**
- **(A)** 130
- **(B)** 124
- **(C)** 136
- **(D)** 118

**Correct Answer:** Option **(A)** — `130`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
130. Tracing directly (j increments before each continue, so the loop always terminates correctly): i=1 contributes 1; i=2 contributes 2; i=3 contributes 3+9=12; i=4 contributes 4+12=16; i=5 contributes 5+15+25=45; i=6 contributes 6+18+30=54. Total = 1+2+12+16+45+54 = 130.

---

## Question ITM-P03-Q49 (Infosys System Engineer ITM Mock Paper 3 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_03_page_06.png

**Pseudocode / Algorithm Stem:**

```
A stack is used (LIFO). What is printed?
```

**Options:**
- **(A)** 39
- **(B)** 33
- **(C)** 42
- **(D)** 30

**Correct Answer:** Option **(A)** — `39`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
39. Stack: push 4,7 → [4,7]. a=pop=7, b=pop=4 → push(7−4=3) → [3]. push(10) → [3,10]. c=pop=10, d=pop=3 → push(13) → [13]. push(3) → [13,3]. e=pop=3, f=pop=13. e×f = 39.

---

## Question ITM-P03-Q50 (Infosys System Engineer ITM Mock Paper 3 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_03_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is the final content of arr? (0-indexed; each statement uses the array's state at that point)
```

**Options:**
- **(A)** [60, 40, 30, 20, 50, 10]
- **(B)** [60, 20, 30, 40, 50, 10]
- **(C)** [10, 40, 30, 20, 50, 60]
- **(D)** [60, 40, 50, 20, 30, 10]

**Correct Answer:** Option **(A)** — `[60, 40, 30, 20, 50, 10]`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
i=0,j=5: swap arr[0],arr[5] → [60,20,30,40,50,10]; i=1,j=3. i<j (1<3): swap arr[1],arr[3] → [60,40,30,20,50,10]; i=2,j=1. Now i<j fails (2<1), loop ends. Final: [60,40,30,20,50,10].

---

## Question ITM-P04-Q46 (Infosys System Engineer ITM Mock Paper 4 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_04_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (% is the modulo operator)
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a % b);
}
print(gcd(252, 105))
```

**Options:**
- **(A)** 21
- **(B)** 42
- **(C)** 7
- **(D)** 14

**Correct Answer:** Option **(A)** — `21`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
21. gcd(252,105): 252%105=42 → gcd(105,42): 105%42=21 → gcd(42,21): 42%21=0 → gcd(21,0)=21.

---

## Question ITM-P04-Q47 (Infosys System Engineer ITM Mock Paper 4 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_04_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (^=XOR, |=OR, &=AND, >>=right shift)
a = 13; b = 25;
c = (a ^ b) & (a | b);
d = c >> 2;
print(c + d)
```

**Options:**
- **(A)** 25
- **(B)** 29
- **(C)** 20
- **(D)** 33

**Correct Answer:** Option **(A)** — `25`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
25. 13=01101, 25=11001. a^b = 10100 = 20. a|b = 11101 = 29. c = 20 & 29 = 10100 = 20. d = 20>>2 = 5. c+d = 20+5 = 25.

---

## Question ITM-P04-Q48 (Infosys System Engineer ITM Mock Paper 4 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_04_page_06.png

**Pseudocode / Algorithm Stem:**

```
How many swaps are performed in total?
arr = [5, 2, 8, 1, 9, 3]
swaps = 0
for i = 0 to 4
    for j = 0 to 4 - i
        if (arr[j] > arr[j+1])
            temp = arr[j]
            arr[j] = arr[j+1]
            arr[j+1] = temp
            swaps = swaps + 1
        end if
    end for
end for
print(swaps)
```

**Options:**

- **(A)** 7
- **(B)** 8
- **(C)** 6
- **(D)** 9

**Correct Answer:** Option **(A)** — `7`

**Step-by-Step Dry-Run Execution Trace & Analysis:**

7. Full bubble-sort trace: pass i=0 gives 3 swaps (array becomes [2,5,1,8,3,9]); pass i=1 gives 2 more swaps ([2,1,5,3,8,9]); pass i=2 gives 2 more swaps ([1,2,3,5,8,9], now sorted); passes i=3 and i=4 give 0 swaps. Total = 3+2+2 = 7.

---

## Question ITM-P04-Q49 (Infosys System Engineer ITM Mock Paper 4 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 6

**Visual Reference Plate:** paper_04_page_06.png

**Pseudocode / Algorithm Stem:**

```
This algorithm reverses the first k elements of a queue using a stack. What is printed?
queue = [1, 2, 3, 4, 5]
stack = []
k = 3
for i = 1 to k
    push(stack, dequeue(queue))
end for
while (stack is not empty)
    enqueue(queue, pop(stack))
end while
for i = 1 to (5 - k)
    enqueue(queue, dequeue(queue))
end for
print(queue)
```

**Options:**

- **(A)** [3, 2, 1, 4, 5]

- **(B)** [1, 2, 3, 4, 5]
- **(C)** [5, 4, 3, 2, 1]
- **(D)** [3, 2, 1, 5, 4]

**Correct Answer:** Option **(A)** — `[3, 2, 1, 4, 5]`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
This is the standard "reverse first k of a queue" algorithm. After moving the first 3 elements through the stack, the queue becomes [4,5,3,2,1]; the final rotation step (dequeue+enqueue twice) moves 4 and 5 to the back, giving [3,2,1,4,5] — the first 3 elements correctly reversed.

---

## Question ITM-P04-Q50 (Infosys System Engineer ITM Mock Paper 4 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 7
**Visual Reference Plate:** paper_04_page_07.png

**Pseudocode / Algorithm Stem:**

```
This algorithm moves all non-zero elements to the front, preserving order, and fills the rest with
zeroes.
What is the final arr?
arr = [3, 0, 4, 0, 5, 0, 2]
pos = 0
for i = 0 to 6
    if (arr[i] != 0)
        arr[pos] = arr[i]
        pos = pos + 1
    end if
end for
while (pos <= 6)
    arr[pos] = 0
    pos = pos + 1
end while
print(arr)
```

**Options:**
- **(A)** [3, 4, 5, 2, 0, 0, 0]
- **(B)** [3, 0, 4, 0, 5, 0, 2]
- **(C)** [3, 4, 5, 2, 0, 0, 2]
- **(D)** [0, 0, 0, 3, 4, 5, 2]

**Correct Answer:** Option **(A)** — `[3, 4, 5, 2, 0, 0, 0]`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
Non-zero elements (3,4,5,2) are copied to the front in order, giving positions 0–3; the while loop then zeroes out positions 4,5,6. Final: [3,4,5,2,0,0,0].

---

## Question ITM-P05-Q46 (Infosys System Engineer ITM Mock Paper 5 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_05_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (This computes the number of grid paths from one corner to another, moving only
right or down.)
int f(int n, int m) {
    if (n == 0 || m == 0) return 1;
    return f(n-1, m) + f(n, m-1);
}
print(f(3, 2))
```

**Options:**
- **(A)** 10
- **(B)** 8
- **(C)** 12
- **(D)** 9

**Correct Answer:** Option **(A)** — `10`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
10. This recursion counts grid paths and equals C(n+m,n). Tracing directly: f(1,1)=2, f(2,1)=3, f(1,2)=3, f(2,2)=6, f(3,1)=4, f(3,2)=f(2,2)+f(3,1)=6+4=10 (also confirmed by code execution).

---

## Question ITM-P05-Q47 (Infosys System Engineer ITM Mock Paper 5 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_05_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (>>=right shift, <<=left shift, &=AND, ^=XOR)
x = 0;
for i = 0 to 7
    if (((45 >> i) & 1) == 1)
        x = x + (1 << (7 - i))
    end if
end for
y = x ^ 45
print(y)
```

**Options:**
- **(A)** 153
- **(B)** 180
- **(C)** 135
- **(D)** 171

**Correct Answer:** Option **(A)** — `153`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
153. 45 = 00101101 in 8 bits. The loop reverses these 8 bits into x, giving x=180 (10110100). y = x^45 = 180^45 = 153 (verified by direct computation).

---

## Question ITM-P05-Q48 (Infosys System Engineer ITM Mock Paper 5 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_05_page_06.png

**Pseudocode / Algorithm Stem:**

```
This is a binary search for target = 50 in a sorted array. How many comparisons are made before
the loop ends (the target is not present)?
arr = [2, 5, 8, 12, 16, 23, 38, 45, 56, 72, 91]
low = 0; high = 10; target = 50; comparisons = 0
while (low <= high)
    mid = (low + high) / 2
    comparisons = comparisons + 1
    if (arr[mid] == target)
        break
    else if (arr[mid] < target)
        low = mid + 1
    else
        high = mid - 1
    end if
end while
print(comparisons)
```

**Options:**
- **(A)** 4
- **(B)** 3
- **(C)** 5
- **(D)** 6

**Correct Answer:** Option **(A)** — `4`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
4. Tracing the search for target=50 (not present): mid=5(23)→low=6; mid=8(56)→high=7; mid=6(38)→low=7; mid=7(45)→low=8, loop ends. That's 4 comparisons (verified by direct trace).

---

## Question ITM-P05-Q49 (Infosys System Engineer ITM Mock Paper 5 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 7
**Visual Reference Plate:** paper_05_page_07.png

**Pseudocode / Algorithm Stem:**

```
This stack trace evaluates a postfix-style expression. What is printed?
push(6); push(2); push(3);
c = pop(); b = pop(); push(b + c);
d = pop(); e = pop(); push(e - d);
push(3); push(8); push(2);
f = pop(); g = pop(); push(g / f);
h = pop(); i = pop(); push(i + h);
j = pop(); k = pop(); push(k * j);
print(pop())
```

**Options:**
- **(A)** 7
- **(B)** 10
- **(C)** 14
- **(D)** 4

**Correct Answer:** Option **(A)** — `7`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
7. This performs the postfix evaluation of "6 2 3 + − 3 8 2 / + ×": (2+3)=5, (6−5)=1, (8/2)=4, (3+4)=7, (1×7)=7.

---

## Question ITM-P05-Q50 (Infosys System Engineer ITM Mock Paper 5 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 7
**Visual Reference Plate:** paper_05_page_07.png

**Pseudocode / Algorithm Stem:**

```
This is Kadane's algorithm for maximum subarray sum. What is printed?
arr = [-2, 3, -1, 5, -3, 4, -2, 6]
maxSoFar = arr[0]; maxEndingHere = arr[0]
for i = 1 to 7
    maxEndingHere = max(maxEndingHere + arr[i], arr[i])
    maxSoFar = max(maxSoFar, maxEndingHere)
end for
print(maxSoFar)
```

**Options:**
- **(A)** 12
- **(B)** 10
- **(C)** 9
- **(D)** 15

**Correct Answer:** Option **(A)** — `12`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
12. Kadane's algorithm finds the maximum subarray sum. Tracing: maxEndingHere/maxSoFar progress to 3, 2→3, 7, 4→7, 8, 6→8, 12→12 (the subarray [5,−3,4,−2,6] sums to 10, but [3,−1,5,−3,4,−2,6] or similarly the running trace peaks at 12 via maxEndingHere reaching 12 at the last element — confirmed by direct code execution).

---

## Question ITM-P06-Q46 (Infosys System Engineer ITM Mock Paper 6 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_06_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (/ is integer division, % is modulo)
int f(int n) {
    if (n <= 1) return n;
    if (n % 2 == 0) return f(n / 2);
    return f(n - 1) + f(n + 1);
}
print(f(11))
```

**Options:**
- **(A)** 5
- **(B)** 4
- **(C)** 6
- **(D)** 3

**Correct Answer:** Option **(A)** — `5`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
5. Building up: f(1)=1, f(2)=f(1)=1, f(4)=f(2)=1, f(3)=f(2)+f(4)=2, f(6)=f(3)=2, f(5)=f(4)+f(6)=3, f(10)=f(5)=3, f(12)=f(6)=2. So f(11)=f(10)+f(12)=3+2=5 (confirmed by code).

---

## Question ITM-P06-Q47 (Infosys System Engineer ITM Mock Paper 6 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_06_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (^=XOR, >>=right shift, &=AND)
n = 13
g = n ^ (n >> 1)
print(g + (g & 5))
```

**Options:**
- **(A)** 12
- **(B)** 11
- **(C)** 16
- **(D)** 14

**Correct Answer:** Option **(A)** — `12`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
12. 13 = $1101_2$; 13>>1 = 6 = $0110_2$; g = 1101 ^ 0110 = $1011_2$ = 11. g & 5 = 1011 & 0101 = 0001 = 1. g + 1 = 12.

---

## Question ITM-P06-Q48 (Infosys System Engineer ITM Mock Paper 6 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_06_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
s = 0
for i = 1 to 5
    for j = 1 to 5
        if ((i + j) % 4 == 0) continue
        if (j > i + 1) break
        s = s + i * j
    end for
end for
print(s)
```

**Options:**
- **(A)** 142
- **(B)** 138
- **(C)** 146
- **(D)** 150

**Correct Answer:** Option **(A)** — `142`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
142. Tracing the loops with the continue (when i+j is a multiple of 4) and break (when j > i+1) gives a total of 142 (confirmed by code).

---

## Question ITM-P06-Q49 (Infosys System Engineer ITM Mock Paper 6 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_06_page_06.png

**Pseudocode / Algorithm Stem:**

```
The following sorts a stack using a temporary stack (the top of a stack is its last element). How
many
times is an element moved from tmp back to st?
st = [3, 1, 4, 2]; tmp = []; moves = 0
while (st is not empty)
    t = pop(st)
    while (tmp is not empty and top(tmp) > t)
        push(st, pop(tmp))
        moves = moves + 1
    end while
    push(tmp, t)
end while
print(moves)
```

**Options:**
- **(A)** 3
- **(B)** 2
- **(C)** 4
- **(D)** 5

**Correct Answer:** Option **(A)** — `3`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
3. Tracing: pop 2 → tmp [2]; pop 4 → tmp [2,4]; pop 1 → move 4 and 2 back (2 moves) → tmp [1]; pop 2 → tmp [1,2]; pop 4 → tmp [1,2,4]; pop 3 → move 4 back (1 move), tmp [1,2,3]; pop 4 → tmp [1,2,3,4]. Total moves = 3.

---

## Question ITM-P06-Q50 (Infosys System Engineer ITM Mock Paper 6 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_06_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (dynamic programming for the minimum number of coins; INF is a very large number)
coins = [1, 3, 4]
dp[0] = 0; dp[1..6] = INF
for i = 1 to 6
    for each c in coins
        if (i >= c)
            dp[i] = min(dp[i], dp[i - c] + 1)
        end if
    end for
end for
print(dp[6])
```

**Options:**
- **(A)** 2
- **(B)** 3
- **(C)** 4
- **(D)** 6

**Correct Answer:** Option **(A)** — `2`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
2. dp = [0, 1, 2, 1, 1, 2, 2], so dp[6] = 2 (3+3). A greedy approach (4+1+1) would give 3, which is why the DP matters.

---

## Question ITM-P07-Q46 (Infosys System Engineer ITM Mock Paper 7 - Q46)

**Section:** Section 4: Pseudo Code | **Page:** 6
**Visual Reference Plate:** paper_07_page_06.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (Operands are evaluated strictly from left to right; x++ and x-- return the old
value
and then change x, while ++x and --x change x first and then return the new value.)
x = 4
y = x++ * 2 + ++x
z = --x + x-- * 2
print(x + y + z)
```

**Options:**
- **(A)** 33
- **(B)** 31
- **(C)** 35
- **(D)** 29

**Correct Answer:** Option **(A)** — `33`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
33. Line 2: x++ returns 4 (x becomes 5); ×2 = 8; ++x makes x = 6 and returns 6; y = 8 + 6 = 14, x = 6.
Line 3: --x makes x = 5 and returns 5; x-- returns 5 (x becomes 4); 5×2 = 10; z = 5 + 10 = 15, x = 4. Result = 4 +
14 + 15 = 33.

---

## Question ITM-P07-Q47 (Infosys System Engineer ITM Mock Paper 7 - Q47)

**Section:** Section 4: Pseudo Code | **Page:** 7
**Visual Reference Plate:** paper_07_page_07.png

**Pseudocode / Algorithm Stem:**

```
What is printed? (/ is integer division)
int g(int n) {
    if (n == 0) return 0;
    if (n % 2 == 1) return g(n - 1) + n;
    return g(n / 2) + n / 2;
}
print(g(10))
```

**Options:**
- **(A)** 14
- **(B)** 17
- **(C)** 15
- **(D)** 12

**Correct Answer:** Option **(A)** — `14`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
14. g(1)=g(0)+1=1; g(2)=g(1)+1=2; g(4)=g(2)+2=4; g(5)=g(4)+5=9; g(10)=g(5)+5 = 14 (confirmed by code).

---

## Question ITM-P07-Q48 (Infosys System Engineer ITM Mock Paper 7 - Q48)

**Section:** Section 4: Pseudo Code | **Page:** 7
**Visual Reference Plate:** paper_07_page_07.png

**Pseudocode / Algorithm Stem:**

```
The graph below is stored as an adjacency list, and this iterative traversal marks a node as
visited only
when it is popped. In what order are the nodes visited?
adj[0] = [1, 2]
adj[1] = [3, 4]
adj[2] = [4, 5]
adj[3] = []
adj[4] = [6]
adj[5] = [6]
adj[6] = []

stack = [0]
while (stack is not empty)
    v = pop(stack)
    if (v is visited) continue
    mark v visited and print v
    for each w in adj[v] (in the order listed)
        push(stack, w)
    end for
end while
```

**Options:**

- **(A)** 0 2 5 6 4 1 3
- **(B)** 0 1 3 4 6 2 5
- **(C)** 0 1 2 3 4 5 6
- **(D)** 0 2 4 6 5 1 3

**Correct Answer:** Option **(A)** — `0 2 5 6 4 1 3`

**Step-by-Step Dry-Run Execution Trace & Analysis:**

Visit 0 and push 1, 2. Pop 2, visit, push 4, 5. Pop 5, visit, push 6. Pop 6, visit. Pop 4, visit (its neighbour 6 is pushed but already visited). Pop 6 (skipped), pop 1, visit, push 3, 4. Pop 4 (skipped), pop 3, visit. Order: 0 2 5 6 4 1 3.

---

## Question ITM-P07-Q49 (Infosys System Engineer ITM Mock Paper 7 - Q49)

**Section:** Section 4: Pseudo Code | **Page:** 7

**Visual Reference Plate:** paper_07_page_07.png

**Pseudocode / Algorithm Stem:**

```
What is printed?
arr = [3, 4, 10, 11, 14, 7, 0]
count = 0
for i = 0 to 6
    for j = i + 1 to 6
        if ((arr[i] + arr[j]) % 7 == 0)
            count = count + 1
        end if
    end for
end for
print(count)
```

**Options:**
- **(A)** 7
- **(B)** 6
- **(C)** 8
- **(D)** 5

**Correct Answer:** Option **(A)** — `7`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
7. Taking each value modulo 7 gives 3, 4, 3, 4, 0, 0, 0. A pair sums to a multiple of 7 when it is (3,4) or (0,0). There are 2×2 = 4 pairs of a 3 with a 4, and C(3,2) = 3 pairs of zeros, so the count is 4 + 3 = 7 (confirmed by code).

---

## Question ITM-P07-Q50 (Infosys System Engineer ITM Mock Paper 7 - Q50)

**Section:** Section 4: Pseudo Code | **Page:** 7
**Visual Reference Plate:** paper_07_page_07.png

**Pseudocode / Algorithm Stem:**

```
This is the 0/1 knapsack dynamic programme. What is printed?
wt  = [2, 3, 4, 5]
val = [3, 4, 5, 6]
W = 5
dp[0..W] = 0
for i = 0 to 3
    for w = W down to wt[i]
        dp[w] = max(dp[w], dp[w - wt[i]] + val[i])
    end for
end for
print(dp[W])
```

**Options:**
- **(A)** 7
- **(B)** 6
- **(C)** 8
- **(D)** 9

**Correct Answer:** Option **(A)** — `7`

**Step-by-Step Dry-Run Execution Trace & Analysis:**
7. The best choice for capacity 5 is items of weight 2 and 3, worth 3 + 4 = 7. The loop runs w downwards, so each item is used at most once.

---